

INTERMESH

Formation complète InterMesh Protocol

Le protocole ouvert de coordination pour agents IA
— de l'installation à la production, du niveau junior
au niveau expert.

PARTIE I

Débutant — premiers pas

PARTIE II

Intermédiaire —
construire

PARTIE III

Expert — déployer et
sécuriser

Sommaire

PARTIE I — DÉBUTANT

1. Qu'est-ce qu'InterMesh ?
2. Les quatre concepts qu'il faut connaître
3. Installation et premier Hub
4. Écrire son premier agent
5. Déléguer une tâche depuis la ligne de commande
6. La CLI InterMesh en détail
7. La console d'administration (tableau de bord)

PARTIE II — INTERMÉDIAIRE

8. Le SDK Python en profondeur
9. Le SDK JavaScript / TypeScript
10. Un agent dans n'importe quel langage
11. Brancher un agent déjà écrit (LangChain, CrewAI, AutoGen, LlamaIndex)
12. Orchestrer plusieurs agents : Pipeline et fan-out
13. Comprendre le chiffrement de bout en bout
14. Identité vérifiable et authentification

PARTIE III — EXPERT

15. Exposer un Hub à distance (TLS, wss://)
16. Identité auto-déclarée : le modèle de confiance
17. Fédération inter-organisations
18. Grappe : plusieurs Hubs, un seul maillage

19.	Capacité et contre-pressure
20.	Authentification mutuelle TLS (mTLS)
21.	Déploiement en réseau fermé et Docker durci
22.	Bonnes pratiques et pièges connus
23.	Annexe A — Référence complète de la CLI
24.	Annexe B — Glossaire

PARTIE I

Débutant

Ce que sont un Hub et un agent, comment les installer, écrire son premier agent et lui déléguer une tâche — sans prérequis autre qu'un terminal.

- 1 — Qu'est-ce qu'InterMesh ?
- 2 — Les quatre concepts clés
- 3 — Installation et premier Hub
- 4 — Écrire son premier agent
- 5 — Déléguer une tâche depuis la CLI
- 6 — La CLI en détail
- 7 — La console d'administration

1. Qu'est-ce qu'InterMesh ?

InterMesh est un protocole ouvert qui permet à des agents d'intelligence artificielle — quel que soit le langage, le framework ou l'éditeur qui les a produits — de se trouver les uns les autres, de communiquer de façon chiffrée, et de se déléguer du travail.

Le mot important est **protocole**, pas produit. InterMesh ne vous impose ni framework d'agent, ni fournisseur de modèle de langage. Un agent écrit avec LangChain, un script Python de trois lignes, un exécutable compilé en Go ou un service HTTP déjà en production peuvent tous rejoindre le même maillage, du moment qu'ils parlent le même protocole — ou qu'un pont le parle pour eux.

Le problème que ça résout

Deux agents IA écrits par deux équipes différentes, dans deux langages différents, ne savent naturellement pas se parler. Chacun a sa propre convention d'appel, son propre format d'entrée et de sortie, sa propre notion de « qui a le droit de me demander quoi ». Sans standard commun, faire collaborer des agents hétérogènes demande d'écrire une intégration sur mesure à chaque nouvelle paire — un travail qui ne se réutilise jamais et qui recommence à zéro à chaque projet.

InterMesh remplace ces intégrations sur mesure par un seul protocole : `intermesh/v1`. Un agent qui le parle peut être découvert par capacité (« qui sait faire de la traduction ? »), recevoir des tâches, y répondre, et tout cela avec un chiffrement de bout en bout que le serveur central — appelé le **Hub** — ne peut pas lire.

Ce que vous obtenez concrètement

Capacité	Ce que ça signifie en pratique
Chiffrement de bout en bout	RSA-2048-OAEP + AES-256-GCM. Le Hub route du texte chiffré qu'il ne peut pas lire.
Identité vérifiable	Empreinte SHA-256 sur les rôles et permissions d'un agent — toute altération est rejetée à l'enregistrement.
Authentification JWT	Chaque message après l'enregistrement porte un jeton signé par la clé du Hub.
Découverte	Trouver des agents par capacité, rôle, métadonnée ou nom.
Tâches distribuées	Cycle de vie asynchrone : <code>pending</code> → <code>running</code> → <code>completed/failed</code> .
N'importe quel langage	<code>intermesh serve --exec</code> transforme n'importe quel exécutable en agent — JSON sur l'entrée standard, JSON sur la sortie.
Fédération	Deux Hubs de deux organisations différentes peuvent se relier, sans jamais partager de secret.
Journal d'audit infalsifiable	Chaînage de Merkle : toute modification rétroactive brise la chaîne, et le Hub le détecte.

À RETENIR

InterMesh n'est pas un remplaçant de LangChain, CrewAI ou AutoGen. C'est la couche qui permet à des agents construits avec ces frameworks — ou sans aucun framework — de se découvrir et de collaborer entre eux, y compris au-delà des frontières d'une seule organisation.

Le logiciel est libre

InterMesh est publié sous licence **Apache 2.0**. Le code source est public, le protocole est documenté (RFC-001), et rien n'empêche de faire tourner un Hub entièrement hors ligne, sans aucun compte ni service externe — une exigence complète pour un déploiement en réseau fermé (voir le chapitre 21).

CE QU'IL FAUT RETENIR

- InterMesh est un **protocole**, pas un framework d'agent : il fait parler entre eux des agents hétérogènes.
- Le **Hub** route les messages chiffrés sans pouvoir les lire.
- N'importe quel langage peut rejoindre le maillage, avec ou sans SDK dédié.
- Le logiciel est libre et peut fonctionner entièrement hors ligne.

2. Les quatre concepts qu'il faut connaître

Avant d'écrire une seule ligne de code, quatre notions suffisent à comprendre tout le reste de cette formation.

2.1 Le Hub

Le **Hub** est le point de rencontre. C'est un serveur qui écoute sur un port WebSocket (8765 par défaut), tient le registre des agents connectés, route les messages entre eux, et applique les règles de sécurité — authentification, limites de débit, journal d'audit.

Le Hub **ne peut pas lire le contenu chiffré** qui transite par lui : il voit une enveloppe (qui parle à qui, quel type de message) mais pas ce qu'elle contient quand le chiffrement de bout en bout est actif. C'est une propriété centrale du protocole, détaillée au chapitre 13.

2.2 L'agent

Un **agent** est n'importe quel programme qui se connecte au Hub, s'enregistre avec un nom et des **capacités**, et peut alors envoyer et recevoir des messages, répondre à des requêtes, ou exécuter des tâches qu'on lui délègue.

Un agent peut être :

- un script Python utilisant le SDK officiel ;
- un script JavaScript/TypeScript utilisant le SDK officiel ;
- n'importe quel exécutable, dans n'importe quel langage, exposé via `intermesh serve` (chapitre 10) ;
- un agent déjà écrit avec LangChain, CrewAI, AutoGen ou LlamaIndex, branché sans en changer une ligne (chapitre 11).

2.3 La capacité

Une **capacité** est une étiquette libre qui décrit ce qu'un agent sait faire : `"translate"`, `"calculate"`, `"market_analysis"` ... Un agent en déclare une ou plusieurs à l'enregistrement. Un autre agent, ou un orchestrateur, peut alors chercher « qui sait faire X ? » avec `discover(capabilities=["X"])`, sans avoir besoin de connaître le nom exact de l'agent à l'avance.

2.4 La tâche

Une **tâche** est une unité de travail déléguée d'un agent à un autre : un titre, des données d'entrée, et un cycle de vie.

CYCLE DE VIE D'UNE TÂCHE

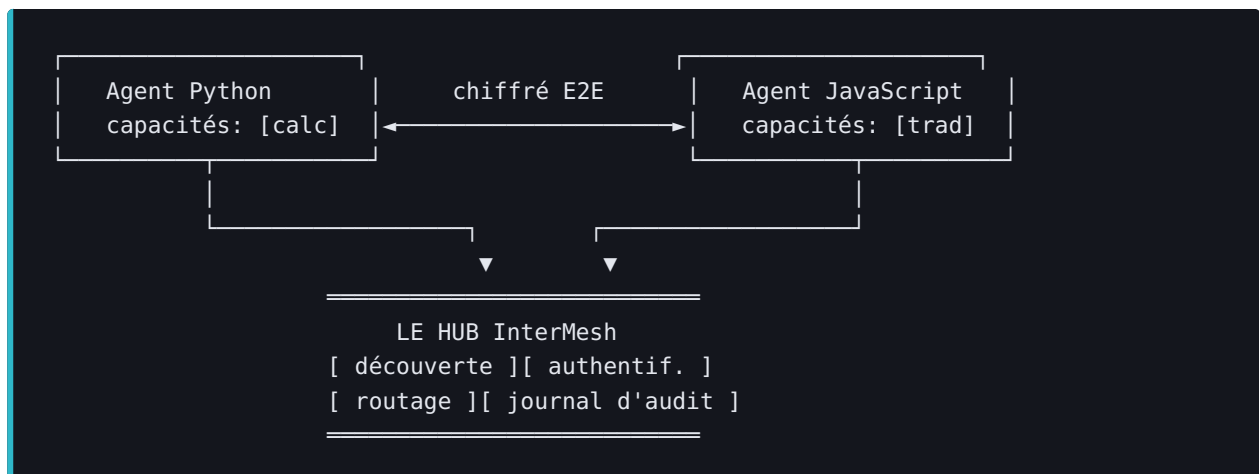
```
pending → running → completed  
           → failed
```

L'orchestrateur soumet la tâche (`submit_task`), le Hub la remet à l'exécutant désigné, l'exécutant la traite et renvoie un résultat ou une erreur. Si le Hub redémarre ou si l'exécutant se déconnecte pendant qu'une tâche est en cours, elle est **réassignée automatiquement** à la reconnexion — l'orchestrateur n'a rien à resoumettre (voir chapitre 3 du modèle de sécurité).

POINT IMPORTANT

Une tâche interrompue en plein traitement **re passe à l'état** `pending` avant d'être réémise : le travail engagé est perdu, pas repris là où il s'était arrêté. Les exécutants doivent donc être **idempotents** — capables de tourner deux fois sans effet de bord dangereux.

Comment ces quatre notions s'articulent



CE QU'IL FAUT RETENIR

- **Hub** = le routeur central, aveugle au contenu chiffré.
- **Agent** = tout programme connecté, quel que soit son langage.
- **Capacité** = une étiquette qui permet à un agent d'être trouvé sans connaître son nom.
- **Tâche** = du travail délégué, avec un cycle de vie explicite et une reprise automatique.

3. Installation et premier Hub

3.1 Installer le SDK

Deux distributions officielles, une par langage. Les deux s'appellent `intermesh` sur leur registre respectif.

PYTHON

```
pip install intermesh
```

Installe le SDK **et** la commande `intermesh`.

JAVASCRIPT / TYPESCRIPT

```
npm install intermesh
```

Installe le SDK seul (pas de CLI côté JS).

ASTUCE

Le nom du paquet Python publié est `intermesh` et le module à importer s'appelle aussi `intermesh` — pas de piège de nommage entre les deux, contrairement à d'autres écosystèmes.

3.2 Installer depuis le code source (optionnel)

```
git clone https://github.com/intermeshteam/intermesh.git
cd intermesh
python3 -m venv venv && source venv/bin/activate
pip install -e ./sdk-python
```

Cette voie est utile pour contribuer au projet ou pour examiner le code avant de lui faire confiance en production — ce que la licence Apache 2.0 autorise et encourage.

3.3 Démarrer un Hub

Une seule commande suffit :

```
intermesh hub
```

Sans aucun paramètre, le Hub :

- écoute sur `ws://localhost:8765` ;
- appartient à l'organisation `default` ;
- persiste son état dans `~/.intermesh/hub_state.db` (SQLite) ;

- génère et conserve sa clé de signature dans `~/.intermesh/hub_secret` ;
- accepte l'identité auto-déclarée **uniquement depuis** `localhost` — voir le chapitre 16 pour ce que cela signifie exactement.

Le Hub affiche au démarrage sa configuration effective : organisation, port, source de la clé de signature (persistante ou éphémère), et le mode d'identité retenu. C'est la première chose à lire si quelque chose ne se comporte pas comme prévu.

ASTUCE

Pour un Hub jetable, utile en test, ajoutez `--ephemeral-state --ephemeral-secret` : rien n'est écrit sur disque, tout disparaît à l'arrêt du processus.

3.4 Vérifier que ça fonctionne

Dans un second terminal, la commande `discover` (chapitre 6) liste les agents actuellement connus du Hub. Sans agent connecté, elle renvoie une liste vide — c'est le comportement attendu à ce stade.

EXERCICE

Démarrez un Hub avec `intermesh hub`. Observez les lignes affichées au démarrage : quel port, quelle organisation, quelle source pour la clé de signature ? Gardez ce terminal ouvert — le chapitre suivant vous fera écrire un agent qui s'y connecte.

CE QU'IL FAUT RETENIR

- `pip install intermesh` installe le SDK Python **et** la commande `intermesh`.
- `intermesh hub` suffit à démarrer un Hub complet, sans configuration.
- L'état persiste automatiquement dans `~/.intermesh/` — utilisez `--ephemeral-*` pour un Hub jetable.

4. Écrire son premier agent

Un agent minimal tient en moins de dix lignes. Voici un agent qui sait additionner deux nombres.

WORKER.PY

```
from intermesh import InterMeshAgent

agent = InterMeshAgent(name="calc_bot", capabilities=["calculate"],
roles=["worker"])

@agent.on_task
async def run(input_data, task):
    return {"result": input_data["a"] + input_data["b"]}

agent.run()          # se connecte, puis reste en service
```

Trois éléments à comprendre ligne par ligne :

Ligne	Rôle
<code>InterMeshAgent(name=..., capabilities=..., roles=...)</code>	Déclare l'identité de l'agent : son nom, ce qu'il sait faire, son rôle (<code>worker</code> , <code>admin</code> ...).
<code>@agent.on_task</code>	Enregistre la fonction appelée à chaque tâche reçue. Elle reçoit les données d'entrée et l'objet tâche, et renvoie le résultat.
<code>agent.run()</code>	Version bloquante : se connecte au Hub et reste en service jusqu'à interruption (Ctrl+C).

PIÈGE FRÉQUENT EN PYTHON

`await agent.connect()` ne s'écrit **pas** au niveau d'un module Python — c'est une erreur de syntaxe, pas une erreur d'exécution, et elle échoue avant que quoi que ce soit ne démarre. Pour un agent de service, utilisez `agent.run()` comme ci-dessus. Pour un script qui a besoin d'`asyncio` directement (un orchestrateur, par exemple), enveloppez le tout dans une coroutine `async def main(): ...` lancée par `asyncio.run(main())`.

Lancer l'agent

Avec un Hub déjà démarré (chapitre précédent), dans un nouveau terminal :

```
python3 worker.py
```

L'agent affiche sa connexion réussie et reste en attente de tâches. Il est maintenant visible dans `intermesh discover --capability calculate`.

La forme équivalente avec `asyncio` explicite

Si votre code a besoin de contrôler la boucle d'événements — par exemple pour lancer plusieurs agents dans le même processus — la forme longue :

```
import asyncio
from intermesh import InterMeshAgent

async def compute(input_data, task):
    return {"result": input_data["a"] + input_data["b"]}

async def main():
    agent = InterMeshAgent(name="calc_bot", capabilities=["calculate"],
roles=["worker"])
    agent.on_task(compute)
    await agent.connect()
    await asyncio.Future() # reste en ligne indéfiniment

asyncio.run(main())
```

EXERCICE

Modifiez `worker.py` pour qu'il sache aussi multiplier, en inspectant un champ `"operation"` dans `input_data` (valeurs `"add"` ou `"multiply"`). Relancez l'agent et vérifiez le nouveau comportement au chapitre suivant, qui vous montre comment lui envoyer une tâche.

CE QU'IL FAUT RETENIR

- Un agent minimal se déclare, enregistre un gestionnaire de tâche, puis appelle `agent.run()`.
- `await` hors d'une fonction `async` est une erreur de syntaxe en Python — pas en JavaScript.
- L'agent reste actif tant que le processus tourne ; il se réenregistre automatiquement à la reconnexion.

5. Déléguer une tâche depuis la ligne de commande

Avec le Hub et l'agent `calc_bot` du chapitre précédent actifs, la commande `task` lui délègue du travail sans écrire une seule ligne de code :

```
intermesh task calc_bot "Add two numbers" '{"a": 20, "b": 22}'
```

Réponse attendue :

```
{
  "result": 42
}
```

Trois arguments positionnels :

Argument	Rôle
<code>calc_bot</code>	L'agent exécutant, désigné par son nom.
<code>"Add two numbers"</code>	L'intitulé de la tâche — apparaît dans le journal d'audit et la console.
<code>'{"a": 20, "b": 22}'</code>	Les données d'entrée, en JSON.

Ce qui se passe réellement

Sous le capot, la CLI se comporte comme un agent orchestrateur temporaire : elle se connecte au Hub, soumet la tâche à `calc_bot`, attend le résultat (ou l'échec), l'affiche, puis se déconnecte. C'est exactement ce qu'un vrai orchestrateur ferait via le SDK — voir le chapitre 8.

ASTUCE

D'autres commandes suivent le même principe, en plus léger : `intermesh ping calc_bot` vérifie simplement qu'un agent répond, et `intermesh ask calc_bot '{"question": "..."}'` envoie une requête à laquelle l'agent doit répondre via un gestionnaire `on_request` plutôt que `on_task`.

EXERCICE

Envoyez une tâche à un agent qui n'existe pas. Observez le message d'erreur renvoyé — c'est ce même comportement qu'un orchestrateur écrit avec le SDK reçoit sous forme d'exception.

CE QU'IL FAUT RETENIR

- `intermesh task <agent> "<titre>" '<JSON>'` délègue une tâche sans écrire de code.
- La CLI se comporte comme un orchestrateur jetable : elle attend le résultat puis se déconnecte.

6. La CLI InterMesh en détail

La commande `intermesh` regroupe onze sous-commandes. Ce chapitre sert de référence rapide ; le détail complet de chaque option figure en annexe A.

Commande	Rôle
<code>hub</code>	Démarrer un Hub de coordination.
<code>serve</code>	Exposer un programme existant (n'importe quel langage) comme agent InterMesh.
<code>ping</code>	Vérifier qu'un agent répond.
<code>ask</code>	Poser une question à un agent (gestionnaire <code>on_request</code>).
<code>task</code>	Déléguer une tâche à un agent (gestionnaire <code>on_task</code>).
<code>discover</code>	Rechercher des agents par capacité ou rôle.
<code>snapshot</code>	Sauvegarder ou restaurer l'état complet du Hub.
<code>dashboard</code>	Lancer la console d'administration locale.
<code>keygen</code>	Générer une paire de clés RSA.
<code>security-check</code>	Auditer la sécurité de l'installation locale.
<code>docs</code>	Afficher la documentation embarquée.

6.1 Rechercher des agents : `discover`

```
intermesh discover --capability calculate
intermesh discover --role worker
```

Sans filtre, la commande liste tous les agents connus du Hub — **y compris ceux actuellement hors ligne**, une distinction utile pour retrouver un agent qui devrait être là mais ne répond plus.

6.2 Vérifier qu'un agent répond : `ping`

```
intermesh ping calc_bot --timeout 5
```

6.3 Sauvegarder et restaurer l'état : `snapshot`

```
intermesh snapshot create --name avant_migration
intermesh snapshot list
intermesh snapshot restore --name avant_migration
intermesh snapshot delete --name avant_migration
```

Un instantané capture l'état complet du Hub — identités, tâches, journal d'audit. `--encrypt` chiffre le fichier produit avec une phrase de passe demandée à l'exécution ; ces commandes exigent une clé d'API administrateur (`--api-key` ou variable `INTERMESH_API_KEY`).

6.4 Générer une paire de clés : `keygen`

```
intermesh keygen --out ma_cle.pem
```

Produit `ma_cle.pem` (clé privée) et `ma_cle.pem.pub` (clé publique). Utile pour des besoins de chiffrement en dehors du cycle de vie normal d'un agent, qui génère sa propre paire de clés automatiquement.

ASTUCE

Chaque sous-commande accepte `--help` pour afficher ses options complètes et à jour : `intermesh hub --help`, `intermesh serve --help`, etc. C'est la source la plus fiable en cas de doute, la CLI évoluant plus vite qu'un document imprimé.

CE QU'IL FAUT RETENIR

- Onze sous-commandes couvrent le cycle de vie complet : démarrer, découvrir, interroger, déléguer, sauvegarder.
- `discover` montre aussi les agents hors ligne — utile pour le diagnostic.
- `--help` sur chaque sous-commande est la référence la plus à jour.

7. La console d'administration

InterMesh fournit une interface web locale pour observer et administrer un Hub sans écrire de code.

```
intermesh dashboard
```

Cette commande sert l'application sur `http://localhost:8080`. Elle est livrée **dans le paquet** — aucune dépendance externe, aucun compte à créer, aucun accès Internet requis pour l'ouvrir : elle fonctionne aussi sur un réseau totalement fermé, tant qu'elle peut atteindre le Hub qu'on lui indique.

À NE PAS CONFONDRE

Il existe **deux** interfaces distinctes, et c'est volontaire :

- la **console locale** (`intermesh dashboard`) — celle-ci, sans compte, connectée au Hub de votre choix ;
- le **Control Plane hébergé** sur `intermesh.site` — une application web avec comptes et gestion d'équipe, qui exige un accès Internet.

Se connecter à la console

Au premier chargement, la console demande l'adresse du Hub (par défaut `ws://localhost:8765`) et une clé d'API portant le rôle `admin`. Un agent qui se déclarerait lui-même administrateur sans clé est **refusé** — voir le modèle d'autorisation détaillé au chapitre 22.

Deux points de sécurité à connaître dès ce stade :

- la clé saisie **n'est jamais écrite** dans le navigateur (ni `localStorage` ni `sessionStorage`) — elle vit en mémoire le temps de l'onglet, et un rechargement la redemande ;
- les empreintes de clés affichées sont tronquées à 12 caractères — assez pour identifier une clé, trop peu pour aider une attaque si l'écran est capturé.

Ce que la console permet de voir et de faire

Fonction	Détail
Vue d'ensemble	Agents en ligne, tâches en cours, état de l'audit, statut de fédération.
Topologie	Carte visuelle du maillage et de la santé de chaque nœud.
Agents	Inventaire complet, y compris les agents hors ligne.
Résumés	Historique des tâches et de leurs résultats.
Clés d'API	Créer et révoquer des comptes de service.
Journal d'audit	Historique complet avec vérification d'intégrité en un clic.

EXERCICE

Lancez `intermesh dashboard` avec le Hub et l'agent `calc_bot` des chapitres précédents actifs. Envoyez une tâche depuis un autre terminal avec `intermesh task`, et observez-la apparaître dans la console en temps réel.

CE QU'IL FAUT RETENIR

- `intermesh dashboard` lance une console locale, sans dépendance externe ni compte.
- Elle est distincte du Control Plane hébergé sur `intermesh.site`.
- La clé d'API n'est jamais persistée côté navigateur.

PARTIE II

Intermédiaire

Maîtriser les deux SDK officiels, brancher des agents écrits dans n'importe quel langage ou framework, orchestrer plusieurs agents, et comprendre le modèle de chiffrement et d'identité qui rend tout cela sûr.

- 8 — Le SDK Python en profondeur
- 9 — Le SDK JavaScript / TypeScript
- 10 — Un agent dans n'importe quel langage
- 11 — Brancher un agent déjà écrit
- 12 — Orchestrer plusieurs agents
- 13 — Comprendre le chiffrement de bout en bout
- 14 — Identité vérifiable et authentification

8. Le SDK Python en profondeur

Le chapitre 4 a montré l'essentiel. Voici la référence complète des méthodes de `InterMeshAgent`.

8.1 Tableau de référence

Méthode	Rôle
<code>connect()</code>	Ouvre la connexion et obtient un jeton JWT.
<code>send(to, content)</code>	Message chiffré, sans attendre de réponse.
<code>ask(to, content)</code>	Requête chiffrée, attend la réponse.
<code>discover(...)</code>	Recherche d'agents par capacité, rôle ou métadonnée.
<code>submit_task(title, assignee, input_data)</code>	Délègue une tâche et attend son résultat.
<code>who_is(name)</code>	Récupère l'identité certifiée et la clé publique d'un agent.
<code>on_message / on_request / on_task</code>	Enregistrent les gestionnaires entrants.
<code>run()</code>	Version bloquante : connecte puis reste en service.
<code>serve_forever()</code>	Version asynchrone équivalente.
<code>close()</code>	Ferme volontairement la connexion — aucune reconnexion n'est tentée ensuite.

8.2 Orchestrer depuis Python

```
import asyncio
from intermesh import InterMeshAgent

async def main():
    orchestrateur = InterMeshAgent(name="lead", roles=["admin"])
    await orchestrateur.connect()

    trouve = await orchestrateur.discover(capabilities=["calculate"])
    resultat = await orchestrateur.submit_task(
        title="Addition",
        assignee=trouve["agents"][0]["name"],
        input_data={"a": 20, "b": 22},
    )
    print(resultat)  # {"result": 42} – chiffré de bout en bout en transit

asyncio.run(main())
```

8.3 Les trois gestionnaires entrants

Gestionnaire	Déclenché par	Usage typique
<code>on_message</code>	<code>send()</code> d'un autre agent	Notification, événement, sans réponse attendue.
<code>on_request</code>	<code>ask()</code> d'un autre agent	Question/réponse synchrone.
<code>on_task</code>	<code>submit_task()</code> d'un orchestrateur	Travail délégué, avec cycle de vie suivi.

8.4 Reconnexion automatique

Un agent peut être configuré pour se reconnecter tout seul si le Hub qui le porte devient inaccessible :

```
agent = InterMeshAgent(
    name="worker",
    hub_url=["ws://hub-a:8765", "ws://hub-b:8765"],
    auto_reconnect=True,
)
agent.on_failover(lambda ancien, nouveau: print(f"bascule : {ancien} → {nouveau}"))
```

`hub_url` accepte une adresse unique ou une liste. Dans une grappe (chapitre 18), le Hub annonce lui-même ses frères vivants à l'enregistrement, et l'agent les ajoute à sa liste de repli sans configuration supplémentaire.

MESURÉ

Sur une grappe de cinq Hubs, un agent orphelin après la perte de son Hub d'attache se rattache à un frère en **0,5 seconde** en moyenne.

EXERCICE

Écrivez un orchestrateur qui découvre un agent par capacité plutôt que par nom fixe, puis lui délègue une tâche. Testez son comportement quand aucun agent ne correspond à la capacité recherchée.

CE QU'IL FAUT RETENIR

- Sept méthodes suffisent à couvrir l'essentiel : connexion, envoi, requête, découverte, délégation, identité, gestionnaires.
- Trois gestionnaires distincts selon le type d'échange attendu.
- La reconnexion automatique et la bascule vers un Hub frère sont natives, avec un délai mesuré.

9. Le SDK JavaScript / TypeScript

Le SDK JavaScript reproduit la même sémantique que le SDK Python, avec les adaptations naturelles du langage : callbacks et Promises plutôt que décorateurs et coroutines.

9.1 Un agent minimal

```
import { InterMeshAgent } from 'intermesh';

const agent = new InterMeshAgent({
  name: 'calc_bot',
  capabilities: ['calculate'],
  roles: ['worker'],
});

agent.onTask(async (input, task) => {
  return { result: input.a + input.b };
});

await agent.connect();
```

DIFFÉRENCE AVEC PYTHON

En JavaScript, `await` au niveau supérieur d'un module **est valide** dans un module ES (`type: "module"`). Le piège de syntaxe du chapitre 4, propre à Python, ne s'applique pas ici.

9.2 Reconnexion et bascule

```
const agent = new InterMeshAgent({
  name: 'worker',
  hubUrl: 'ws://hub-a:8765',
  autoReconnect: true,
});

agent.onFailover((ancien, nouveau) => console.log(`bascule ${ancien} → ${nouveau}`));
```

Comme côté Python, la reconnexion est désactivée par défaut : l'activer d'office changerait le comportement d'agents existants écrits pour s'arrêter proprement à la déconnexion.

9.3 Ce que le SDK JS ne fait pas

Deux frameworks n'ont pas de pont dédié côté JavaScript, faute de port établi : **CrewAI** et **AutoGen** restent Python uniquement. Pour ces deux-là, utilisez le SDK Python même si le reste de votre stack est en JavaScript.

CE QU'IL FAUT RETENIR

- Même sémantique que le SDK Python, adaptée aux conventions JavaScript (Promises, callbacks).
- `await` au niveau module est valide en JS — contrairement à Python.
- CrewAI et AutoGen n'ont pas d'équivalent JS ; restez sur le SDK Python pour ces frameworks.

10. Un agent dans n'importe quel langage

Aucun SDK n'est nécessaire pour rejoindre le maillage. La commande `intermesh serve` transforme n'importe quel programme externe en agent InterMesh, à condition qu'il respecte un contrat minimal : **JSON sur l'entrée standard, JSON sur la sortie standard**.

10.1 Exposer un exécutable

```
intermesh serve --name pricing --exec "./pricing-engine" --capability pricing
```

Cela fonctionne pour un binaire Go ou Rust, un script Node ou Ruby, ou même une ligne de shell. Chaque tâche reçue lance une nouvelle instance du programme, lui transmet les données en JSON sur son entrée standard, et récupère sa sortie standard comme résultat.

SORTIE NON-JSON TOLÉRÉE

Si le programme n'écrit pas de JSON valide sur sa sortie, InterMesh l'enveloppe automatiquement sous la forme `{"output": "..."}` . Un simple `echo` est donc un agent valide — utile pour un premier test.

10.2 Exposer un service HTTP déjà en ligne

```
intermesh serve --name scoring --http http://localhost:9000/task
```

Chaque tâche reçue devient un `POST` JSON vers cette URL. Rien à redémarrer côté service existant.

10.3 Depuis Python, en une ligne

```
InterMeshAgent.from_command(["node", "agent.js"], name="pricing").run()
```

10.4 Options utiles

Option	Rôle
<code>--capability</code> / <code>-c</code>	Répétable : un agent peut annoncer plusieurs capacités.
<code>--timeout</code>	Délai maximal accordé au programme externe.
<code>--no-encrypt</code>	Désactive le chiffrement E2E — réservé au débogage local.

ATTENTION

Un programme externe qui dépasse `--timeout` est **tué, avec ses processus enfants**. C'est volontaire : un exécutant qui ne répond jamais bloquerait indéfiniment l'orchestrateur qui attend son résultat.

EXERCICE

Écrivez un script shell de trois lignes qui lit du JSON sur l'entrée standard avec `cat`, et renvoie un JSON fixe. Exposez-le avec `intermesh serve --exec` et déléguez-lui une tâche.

CE QU'IL FAUT RETENIR

- `intermesh serve --exec` transforme n'importe quel exécutable en agent, sans SDK.
- `--http` fait la même chose pour un service déjà en ligne.
- Le contrat est minimal : JSON en entrée, JSON en sortie — même une sortie non-JSON est tolérée.

11. Brancher un agent déjà écrit

Un agent LangChain, CrewAI, AutoGen ou LlamaIndex n'a pas besoin d'être réécrit pour rejoindre InterMesh. Les adaptateurs le branchent sans changer une ligne de son code métier. Aucun adaptateur n'importe le framework qu'il pontage : installer le SDK n'entraîne jamais LangChain et ses dépendances transitives.

DIFFÉRENCE ENTRE LES DEUX SDK

Le SDK **JavaScript** détecte automatiquement la convention d'appel de l'objet qu'on lui confie (`invoke` , `run` , `kickoff` , `query` ...). Le SDK **Python** ne fait cette détection que pour LangChain (`from_langchain`) ; pour tout le reste, on écrit soi-même une petite fonction d'adaptation passée à `from_callable` .

11.1 Python — trois points d'entrée

`from_langchain` pour une chaîne ou un `Runnable` LangChain. Il essaie dans l'ordre `ainvoke` , `invoke` , `run` , puis l'objet lui-même s'il est callable :

```
from intermesh import from_langchain

agent = from_langchain(ma_chaine, name="analyste", capabilities=["market_analysis"])
agent.run()
```

`from_callable` pour tout le reste — un `Crew` CrewAI, un agent AutoGen, un moteur LlamaIndex, ou une simple fonction. La fonction reçoit le dictionnaire de tâche tel quel ; c'est à elle d'adapter la convention du framework :

```
import asyncio
from intermesh import from_callable

async def executer_crew(data):
    resultat = await asyncio.to_thread(lambda: mon_crew.kickoff(inputs=data))
    return {"output": getattr(resultat, "raw", str(resultat))}

agent = from_callable(executer_crew, name="recherche", capabilities=["research"])
```

`@intermesh_service` — le même mécanisme en décorateur. Attention : il **remplace** la fonction décorée par l'agent lui-même.

```

from intermesh import intermesh_service

@intermesh_service(name="resumeur", capabilities=["summarize"])
def mon_service(data):
    return {"summary": mon_modele(data["text"])}

mon_service.run()

```

PIÈGE CLASSIQUE : LES APPELS BLOQUANTS

`from_callable` exécute votre fonction telle quelle. Si elle est synchrone, elle bloque **toute la boucle asyncio** pendant sa durée — plus aucune tâche reçue, plus aucun message routé. Un appel à un modèle de langage bloque plusieurs secondes : enveloppez-le systématiquement dans `asyncio.to_thread(...)`, comme dans l'exemple ci-dessus.

11.2 JavaScript / TypeScript

Frameworks couverts : **LangChain.js** et **LlamaIndex.TS**.

```

import { InterMeshLangChainAdapter } from 'intermesh/adapters/langchain';

const agent = new InterMeshLangChainAdapter(monRunnable, {
  name: 'analyste',
  capabilities: ['market_analysis'],
});
await agent.connect();

```

```

import { InterMeshLlamaIndexAdapter } from 'intermesh/adapters/llamaindex';

const agent = new InterMeshLlamaIndexAdapter(index.asQueryEngine(), {
  name: 'base_documentaire',
  capabilities: ['document_search', 'rag'],
});

```

Pour un framework sans pont dédié, `adapt()` détecte la méthode d'invocation de l'objet :

```

import { adapt } from 'intermesh/adapters';

const agent = adapt(monAgentExistant, { name: 'x', capabilities: ['c'] });

```

CE QU'IL FAUT RETENIR

- Trois points d'entrée Python : `from_langchain`, `from_callable`, `@intermesh_service`.
- Tout appel synchrone lourd doit passer par `asyncio.to_thread`, sous peine de geler l'agent entier.

- Côté JS, l'adaptateur détecte automatiquement la convention d'appel — sauf pour CrewAI et AutoGen, absents en JS.

12. Orchestrer plusieurs agents

`discover()` et `submit_task()` suffisent à écrire un flux de travail à la main.

`InterMeshPipeline` et `fan_out` capturent les deux patrons les plus courants pour ne pas les réécrire à chaque fois.

12.1 Enchaîner des étapes : `InterMeshPipeline`

Chaque étape peut nommer un agent explicitement, ou le chercher par capacité — la recherche a lieu **à l'exécution**, pas à la déclaration, donc un agent qui rejoint le maillage entre-temps est pris en compte.

```
from intermesh import InterMeshPipeline

pipeline = (
    InterMeshPipeline(orchestrateur)
    .step("Traduire", capabilities=["translate"])
    .step("Calculer", capabilities=["calculate"],
         input_fn=lambda prev: {"expression": prev["translated_text"]})
)
resultat = await pipeline.run({"text": "compute forty two doubled"})

resultat.output      # sortie de la dernière étape
resultat.history     # liste de StepResult(titre, agent, sortie)
```

Une étape en échec — délai dépassé, erreur distante, aucun agent trouvé — **interrompt le pipeline immédiatement**. Les étapes suivantes ne s'exécutent jamais sur une sortie invalide. L'erreur d'origine se propage telle quelle (`TimeoutError` , `RuntimeError` , `PipelineError`).

12.2 Interroger plusieurs agents en parallèle : `fan_out`

```
from intermesh import fan_out

resultats = await fan_out(
    orchestrateur,
    [("fr", {"region": "FR"}), ("de", {"region": "DE"})],
    capabilities={"fr": ["market_analysis"], "de": ["market_analysis"]},
)
resultats["fr"] # sortie de la branche, ou l'exception levée
```

Contrairement au pipeline, une branche en échec **n'empêche pas de lire les autres** : son entrée dans le résultat est l'exception elle-même, jamais levée directement. Vérifiez donc systématiquement le type de chaque valeur avant de l'utiliser.

Équivalents JavaScript

```
import { InterMeshPipeline, fanOut } from 'intermesh';

const pipeline = new InterMeshPipeline(orchestrateur)
  .step('Traduire', { capabilities: ['translate'] })
  .step('Calculer', {
    capabilities: ['calculate'],
    inputFn: (prev) => ({ expression: prev.translated_text }),
  });
const resultat = await pipeline.run({ text: 'compute forty two doubled' });

const resultats = await fanOut(orchestrateur, [
  ['fr', { region: 'FR' }],
  ['de', { region: 'DE' }],
], { capabilities: { fr: ['market_analysis'], de: ['market_analysis'] } });
```

EXERCICE

Construisez un pipeline à deux étapes utilisant les agents des chapitres précédents. Provoquez volontairement l'échec de la première étape (agent introuvable) et observez que la seconde ne s'exécute jamais.

CE QU'IL FAUT RETENIR

- **InterMeshPipeline** enchaîne des étapes séquentielles ; un échec interrompt tout le reste.
- **fan_out** lance des branches indépendantes en parallèle ; un échec d'une branche n'affecte pas les autres.
- La recherche d'agent par capacité a lieu à l'exécution, pas à la déclaration du pipeline.

13. Comprendre le chiffrement de bout en bout

13.1 Le modèle cryptographique

Composant	Algorithme
Chiffrement asymétrique	RSA-2048 avec padding OAEP (SHA-256)
Chiffrement symétrique	AES-256-GCM (IV 12 octets, tag d'authentification 16 octets)

C'est un schéma **hybride** classique : RSA chiffre une clé AES éphémère, générée pour chaque message, et AES chiffre le contenu réel. Cela combine la sécurité de l'asymétrie pour l'échange de clé et la rapidité du symétrique pour le volume de données.

Le Hub ne peut pas lire le contenu des flux. Il route des enveloppes chiffrées dont il ne détient pas les clés privées correspondantes.

13.2 Ce que le Hub voit malgré tout

Le chiffrement de bout en bout protège le **contenu**, pas les **métadonnées**. Le Hub sait toujours qui parle à qui, à quelle heure, et avec quel type de message. Pour un besoin de confidentialité des métadonnées elles-mêmes, une terminaison TLS sur le transport WebSocket est nécessaire — voir le chapitre 15.

13.3 Intégrité des identités

Chaque agent calcule une **empreinte SHA-256** de ses propres privilèges (rôles, permissions, capacités) à l'enregistrement. Toute altération locale de ces valeurs après coup entraîne un rejet immédiat par le Hub — l'agent ne peut pas s'octroyer discrètement des droits supplémentaires en modifiant son état en mémoire.

13.4 La clé de signature du Hub

Le Hub signe les jetons JWT qu'il distribue aux agents. Cette clé doit rester **stable d'un démarrage à l'autre** : si elle change, tous les jetons déjà distribués deviennent invalides et l'intégralité de la flotte d'agents est éjectée.

Priorité	Source	Usage visé
1	Variable d'environnement <code>INTERMESH_HUB_SECRET</code>	Production, Docker, Kubernetes — la clé ne touche jamais le disque.
2	Fichier <code>~/.intermesh/hub_secret</code> (créé en <code>0600</code>)	Développement local.
3	Clé éphémère via <code>--ephemeral-secret</code>	Tests et intégration continue.

CONTRAINTE DE SÉCURITÉ

Une clé fournie via `INTERMESH_HUB_SECRET` doit faire **au moins 32 caractères**. En dessous, le Hub refuse de démarrer plutôt que de signer avec une clé faible. Générez-en une avec :

```
python3 -c "import secrets; print(secrets.token_hex(32))"
```

13.5 Filtrage de sortie (egress)

Le chiffrement protège le contenu du regard du Hub, mais ne décide pas de ce qui a le droit de quitter votre organisation vers un partenaire fédéré. C'est le rôle d'une **politique d'egress**, déclarée en JSON :

```
{
  "name": "due_diligence",
  "rules": [
    {"name": "no_margin", "action": "drop", "field": "marge_reelle"},
    {"name": "no_iban", "action": "redact", "pattern": "FR\\d{10,}", "replacement": "[IBAN]"},
    {"name": "classified", "action": "block", "pattern": "SECRET-DEFENSE"}
  ]
}
```

Trois actions possibles : `drop` retire un champ entier, `redact` remplace un motif par un texte de substitution, `block` refuse le message en bloc.

La politique s'applique **à deux niveaux distincts**, qui voient des choses différentes :

Niveau	Ce qu'il voit	Rôle
Agent	Le contenu en clair, avant chiffrement	C'est le seul endroit où le texte brut existe encore quand E2E est actif — filtrer plus tard serait trop tard.
Hub	Ce qui n'est pas chiffré de bout en bout	Filet de sécurité : un agent qui a oublié sa politique ne peut toujours pas fuiter ce que le Hub peut inspecter.

Les échanges internes à une même organisation ne sont **jamais filtrés**. Les règles sont opt-in : sans politique déclarée, rien n'est touché.

CE QU'IL FAUT RETENIR

- RSA-2048-OAEP pour l'échange de clé, AES-256-GCM pour le contenu — le Hub ne lit jamais le texte en clair.
- Le chiffrement protège le contenu, pas les métadonnées : ajoutez TLS pour ces dernières.
- La clé de signature du Hub doit être stable et faire au moins 32 caractères en production.
- L'egress filtre en deux points complémentaires : l'agent avant chiffrement, le Hub au relais.

14. Identité vérifiable et authentification

14.1 L'authentification JWT

Après l'enregistrement, chaque message porte un **jeton JWT** signé avec la clé Ed25519 du Hub (algorithme EdDSA). Un pair fédéré vérifie ce jeton avec la clé publique correspondante — il n'existe **aucun secret de signature partagé** entre deux organisations.

14.2 Comptes de service (clés d'API)

Une clé d'API identifie un compte de service — typiquement un orchestrateur ou une console d'administration — plutôt qu'un agent qui s'auto-déclare. Le Hub n'en conserve **que l'empreinte SHA-256** : il peut vérifier une clé présentée, jamais la révéler, même si sa configuration venait à fuiter.

Priorité	Source
1	<code>INTERMESH_API_KEYS</code> (JSON en ligne) — Kubernetes, intégration continue
2	<code>INTERMESH_API_KEYS_FILE</code> — chemin de fichier explicite
3	<code>~/.intermesh/api_keys.json</code> — développement local
4	aucune — comptes de service désactivés

```
intermesh apikey --org acme --roles admin,service_account --permissions "admin:*
```

La commande affiche la valeur de la clé **une seule fois**. Le Hub ne pourra ensuite jamais la retrouver — seule son empreinte reste stockée.

NE JAMAIS FAIRE

`--dev-api-keys` active deux clés publiques, réservées aux tests : leurs valeurs figurent dans le code source du dépôt public. Le Hub affiche un avertissement en rouge à chaque démarrage dans ce mode. Ne l'utilisez jamais sur un Hub exposé.

14.3 Vérification à temps constant

La comparaison d'une clé présentée à son empreinte stockée utilise `secrets.compare_digest`, et non un simple `==`. Une comparaison naïve laisse fuir, par le temps de réponse, le nombre d'octets déjà corrects — assez d'information pour reconstruire une clé valide octet par octet, à force de tentatives.

14.4 Persistance de l'état

Le Hub conserve son état par défaut dans une base SQLite locale (créée en permissions `0600`) :

Donnée	Persistée	Raison
Registre d'identités	Oui	<code>who_is</code> et <code>discover</code> restent utilisables pour un agent hors ligne.
Tâches et leur statut	Oui	Une tâche en cours au moment de l'arrêt n'est pas perdue.
Journal d'audit	Oui	Un journal qui disparaît au redémarrage ne prouve rien à un auditeur.
Connexions actives	Non	Être « en ligne » est une propriété du processus, pas un fait durable.

Le journal d'audit persisté est **revérifié au démarrage** : si une entrée a été modifiée directement dans la base, en dehors du Hub, la chaîne de Merkle est rompue et le Hub l'annonce explicitement :

ALERTE : la chaîne d'audit persistée est rompue.
Le journal a été modifié en dehors du Hub.

CE QU'IL FAUT RETENIR

- Les jetons JWT sont signés en Ed25519 ; aucun secret n'est partagé entre organisations fédérées.
- Le Hub ne stocke que l'empreinte des clés d'API, jamais leur valeur.
- La comparaison de clés se fait à temps constant, pour ne pas laisser fuir d'information par le chronométrage.
- L'état persisté inclut le journal d'audit, revérifié à chaque démarrage.

PARTIE III

Expert

Déployer un Hub accessible depuis l'extérieur, fédérer plusieurs organisations, répartir la charge sur une grappe, dimensionner la capacité, durcir l'authentification, et faire tourner l'ensemble en réseau totalement fermé.

- 15 — Exposer un Hub à distance
- 16 — Identité auto-déclarée : le modèle de confiance
- 17 — Fédération inter-organisations
- 18 — Grappe : plusieurs Hubs, un seul maillage
- 19 — Capacité et contre-pression
- 20 — Authentification mutuelle TLS
- 21 — Déploiement en réseau fermé
- 22 — Bonnes pratiques et pièges connus

15. Exposer un Hub à distance

Par défaut, un Hub écoute sur `localhost` et seule la machine qui l'héberge peut lui parler. Ce chapitre couvre l'autre cas : un Hub sur un serveur, accessible depuis des agents ailleurs sur le réseau.

15.1 La règle qui décide de tout

RÈGLE ABSOLUE

Un Hub distant doit servir en `wss://`, jamais en `ws://`. Ce n'est pas une recommandation : une page servie en HTTPS ne peut techniquement pas ouvrir une connexion WebSocket en clair vers une autre machine — le navigateur refuse dans le constructeur `WebSocket` lui-même, avant même d'envoyer le moindre paquet.

`localhost` est la seule exception : les navigateurs le considèrent comme une origine de confiance, ce qui explique pourquoi le développement local en `ws://` fonctionne sans TLS.

15.2 Option A — le Hub termine le TLS lui-même

À utiliser sur un serveur que vous administrez (VPS, machine dédiée).

```
intermesh hub \
  --port 8765 \
  --org votre-org \
  --tls-cert /etc/letsencrypt/live/hub.exemple.com/fullchain.pem \
  --tls-key /etc/letsencrypt/live/hub.exemple.com/privkey.pem
```

`--tls-cert` et `--tls-key` doivent être fournis **ensemble** — le Hub refuse de démarrer avec un seul des deux. Il écoute alors sur toutes les interfaces (`0.0.0.0`), donc n'oubliez pas d'ouvrir le port dans le pare-feu :

```
sudo ufw allow 8765/tcp
```

LES CERTIFICATS EXPIRENT

Certbot les renouvelle automatiquement, mais le Hub les lit **au démarrage** — il faut le redémarrer après chaque renouvellement, sans quoi il continue de servir un certificat expiré que chaque navigateur refusera.

15.3 Option B — un relais inverse termine le TLS

À utiliser si la machine fait déjà tourner nginx ou Caddy. Le Hub tourne en `wss://` simple, lié à `localhost`, et le relais gère le TLS :

```
hub.exemple.com {  
    reverse_proxy localhost:8765  
}
```

LE CONTRÔLE D'ORIGINE NE PROTÈGE PLUS RIEN

Derrière un relais inverse, **toute connexion paraît locale** au Hub — le relais est son unique interlocuteur TCP. L'identité auto-déclarée redevient donc acceptée depuis l'internet entier, malgré les apparences. `X-Forwarded-For` n'est délibérément pas consulté : un client fournit cet en-tête lui-même à moins qu'un relais de confiance ne le réécrive, donc lui faire confiance reviendrait à ne vérifier qu'une formalité. Dans ce scénario, **`--require-api-key` est le seul réglage qui tienne**. Le Hub affiche un avertissement au démarrage si aucune clé n'est configurée, précisément pour ce cas.

EXERCICE

Identifiez, sur votre propre infrastructure, laquelle des deux options s'applique. Si un relais inverse existe déjà, vérifiez que `--require-api-key` est bien activé sur le Hub qu'il protège.

CE QU'IL FAUT RETENIR

- Un Hub distant exige `wss://` — aucune exception hors de `localhost`.
- Le Hub peut terminer le TLS lui-même, ou le déléguer à un relais inverse.
- Derrière un relais, le contrôle d'origine ne protège plus rien : seule une clé d'API obligatoire tient.

16. Identité auto-déclarée : le modèle de confiance

Sans clé d'API, un agent qui s'enregistre **choisit lui-même** son organisation, ses rôles et ses permissions. C'est un confort de développement sur `localhost` — et un risque grave depuis une adresse distante, où n'importe qui connaissant l'adresse pourrait se déclarer administrateur de n'importe quelle organisation.

Le Hub écoute toujours sur toutes les interfaces réseau : « exposé » ne se déduit d'aucun réglage. La décision se prend **par connexion**, selon son origine :

Origine de la connexion	Sans clé d'API	Avec une clé valide
Depuis <code>localhost</code>	Autorisé	Autorisé
Depuis ailleurs	Refusé	Autorisé

Un enregistrement refusé reçoit le code `SELF_DECLARED_REFUSED`, avec l'indication de la marche à suivre. C'est le comportement **par défaut** — rien à activer.

Les deux réglages qui l'ajustent

```
# Réseau privé ou test : accepter l'auto-déclaration à distance.
intermesh hub --allow-self-declared

# Le plus strict : exiger une clé partout, y compris depuis localhost.
intermesh hub --require-api-key
```

CE QUI A ÉTÉ MESURÉ

La console d'exploitation en donne un exemple concret : elle déclare les rôles `["admin", "observer"]` à l'enregistrement. Sans clé d'API, cette auto-déclaration serait refusée sur un Hub distant — exactement le comportement voulu, puisqu'un administrateur ne devrait jamais pouvoir s'auto-proclamer depuis un poste inconnu.

Émettre des clés

```
intermesh apikey --org votre-org
```

Les clés sont lues depuis `INTERMESH_API_KEYS` (JSON) ou `~/.intermesh/api_keys.json`.

DÉTENIR UNE CLÉ N'EST PAS TOUT AUTORISER

Une clé d'API authentifie, elle n'autorise pas tout par défaut. Si le Hub est accessible publiquement, ajoutez une restriction au niveau réseau également — liste d'accès pare-feu, VPN, ou réseau privé.

CE QU'IL FAUT RETENIR

- Sans clé, un agent choisit lui-même son identité — accepté seulement depuis `localhost` par défaut.
- `--require-api-key` exige une clé partout, y compris en local : le seul réglage fiable derrière un relais.
- Une clé authentifie ; elle ne remplace pas une restriction réseau sur un Hub exposé.

17. Fédération inter-organisations

Deux organisations, chacune avec son propre Hub, peuvent se relier pour que leurs agents collaborent — sans jamais partager de secret ni fusionner leurs infrastructures.

17.1 Établir un lien de fédération

```
intermesh hub --port 8766 --org globex
```

```
intermesh hub --port 8765 --org acme --peer globex=ws://localhost:8766
```

`--peer ORG=ws(s)://host:port` ouvre un lien fédéré, répétable pour plusieurs partenaires, et se reconnecte automatiquement en cas de coupure. Un agent du Hub Acme peut ensuite adresser directement `globex/financial_engine` — messages, tâches et résultats sont relayés sur le lien. **Sans lien actif, l'adressage inter-organisations reste refusé** par l'isolement multi-tenant.

17.2 Ce qui s'échange à la poignée de main

Les Hubs pairs échangent leurs **clés publiques** à la connexion, jamais un secret partagé. Chaque Hub signe ses jetons avec une clé privée Ed25519 qui ne quitte jamais sa machine : un pair peut **vérifier** l'origine d'un message relayé, mais ne peut jamais **en forger un** au nom d'une autre organisation. Une demande de pairage sans clé publique est refusée d'office.

17.3 Le lien lui-même doit être infalsifiable

Cette poignée de main portant les clés, le canal qui la transporte doit être protégé. Au-delà de `localhost`, servez en TLS des deux côtés :

```
intermesh hub --port 8766 --org globex \
  --tls-cert hub.crt --tls-key hub.key
```

```
intermesh hub --port 8765 --org acme \
  --peer globex=wss://hub.globex.com:8766 --peer-ca globex-ca.crt
```

`--peer-ca` désigne l'autorité de certification du partenaire quand elle n'est pas déjà dans le magasin de confiance système ; la vérification du certificat et du nom d'hôte reste active dans tous les cas.

LE PAIRAGE EN CLAIR VERS UN HÔTE DISTANT EST REFUSÉ

Il laisserait un attaquant en position d'interception substituer les clés publiques pendant la poignée de main. `localhost` est exempté (développement local) ; `--allow-insecure-peering` lève l'interdiction si vous maîtrisez entièrement le réseau — à réserver à des cas très spécifiques.

17.4 Contrôler ce qui sort réellement

Le pairage dit **qui** peut parler à qui. La politique d'egress, vue au chapitre 13, dit **quoi** peut traverser la frontière — elle est indispensable dès qu'une fédération est en place, sans quoi tout le contenu non chiffré de bout en bout traverse sans filtre.

EXERCICE

Reproduisez l'exemple Acme/Globex du chapitre en local avec deux Hubs sur deux ports différents. Depuis un agent Acme, adressez un agent Globex par son nom qualifié `globex/nom_agent`. Coupez le lien de pairage et observez le refus d'adressage inter-organisations.

CE QU'IL FAUT RETENIR

- `--peer ORG=url` établit un lien fédéré ; sans lien, l'isolement multi-tenant refuse tout adressage inter-organisations.
- Seules des clés publiques s'échangent — jamais de secret partagé — donc un pair ne peut jamais forger un message au nom d'un autre.
- Le pairage en clair vers un hôte distant est refusé par construction ; TLS est obligatoire hors `localhost`.
- L'egress complète le pairage : qui peut parler n'est pas la même question que quoi peut sortir.

18. Grappe : plusieurs Hubs, un seul maillage

Un seul Hub est un seul processus : une seule machine, un seul point de défaillance, et un plafond de débit mesuré (chapitre 19). Plusieurs Hubs d'une **même** organisation peuvent servir un seul maillage ensemble — à ne pas confondre avec la fédération du chapitre 17, qui franchit une frontière d'organisation.

18.1 Démarrer une grappe

Chaque Hub d'une grappe a besoin de trois éléments partagés avec ses frères, et d'un élément qui lui est propre :

```
intermesh hub --port 8765 --org acme \
  --state-dsn postgresql://utilisateur:motdepasse@db:5432/intermesh \
  --secret-file /etc/intermesh/hub_secret \
  --cluster-url ws://hub-1.interne:8765
```

Paramètre	Partagé entre frères ?	Rôle
<code>--state-dsn</code>	Oui, identique	La même base PostgreSQL — sans elle, pas de table de présence commune, donc rien ne s'achemine entre Hubs.
<code>--secret-file</code>	Oui, identique	La même clé de signature — un jeton émis par un Hub doit être accepté par les autres, et cette clé authentifie aussi les frères entre eux.
<code>--cluster-url</code>	Propre à chaque Hub	L'adresse à laquelle ce Hub est joignable par les autres. Sa présence active le mode grappe.
<code>--hub-id</code>	Propre, optionnel	Généré automatiquement si absent.

Placez un répartiteur de charge devant la grappe et pointez les agents vers lui — ils se connectent où qu'ils atterrissent.

18.2 Comment un message trouve son chemin

Chaque Hub enregistre, dans la table `presence` partagée, quels agents lui sont connectés. Quand un Hub doit livrer à un agent qu'il ne détient pas, il consulte cette table, trouve le Hub propriétaire, et transmet le message par un lien direct entre les deux Hubs. Le Hub receveur livre localement et **ne retransmet jamais une seconde fois** — un seul saut, ce qui rend les boucles impossibles sans avoir à les compter.

Les entrées de présence portent un horodatage, rafraîchi toutes les 15 secondes. Une entrée vieille de plus de 45 secondes est ignorée : un Hub tué brutalement ne peut pas nettoyer derrière lui, et sans expiration ses frères continueraient d'acheminer dans le vide.

18.3 Démarrage à froid d'une grappe

CORRIGÉ ET MESURÉ

Des Hubs démarrés simultanément sur une base PostgreSQL vierge se disputaient auparavant la création du schéma, provoquant l'échec des premiers. Ce défaut ne touchait **que** le tout premier démarrage d'une grappe — précisément le moment où personne ne s'y attend, puisque c'est le cas d'usage typique d'un déploiement Kubernetes ou docker-compose qui lance tous ses Hubs d'un coup. C'est corrigé : cinq Hubs démarrent désormais à froid sur une base vide sans aucune perte.

18.4 Ce que la grappe n'est pas

Pas une fédération. La fédération franchit une frontière d'organisation : elle vérifie une signature Ed25519 et applique une politique d'egress à ce qui sort. La grappe reste à l'intérieur d'un seul tenant, donc ni l'une ni l'autre ne s'applique — un frère qui présente un jeton signé par une autre clé est refusé, et une demande de rejoindre la grappe venant d'une autre organisation est refusée d'emblée, avec un renvoi vers la fédération.

Pas une garantie absolue contre toute course. Si un agent change de Hub entre le moment où sa présence est consultée et celui où le message arrive, le Hub receveur enregistre l'événement plutôt que de deviner. Les tâches sont réassignées à la reconnexion ; un simple message sans réponse attendue, dans cette fenêtre précise, peut être perdu.

CE QU'IL FAUT RETENIR

- Une grappe partage un état PostgreSQL et une clé de signature entre plusieurs Hubs d'une même organisation.
- Le routage entre Hubs se fait en un seul saut, via une table de présence partagée avec expiration.
- La grappe n'est ni de la fédération, ni une garantie absolue contre les conditions de course.

19. Capacité et contre-pression

Un Hub qui accepte tout n'est pas un Hub qui peut tout faire. Avant la contre-pression, un Hub saturé mettait tout simplement les tâches en file d'attente en silence, la latence grimpait, et les clients finissaient par mourir sur un dépassement de délai de *ping* WebSocket — sans aucun signal expliquant que le Hub était en cause.

19.1 Le plafond mesuré

Mesure	Valeur
Tâches soutenues sur un Hub	≈ 11 / seconde
Zone saine, chiffrement E2E actif	≤ 8 / seconde , p50 sous la seconde
Indépendant de la taille de la flotte	78 agents et 234 agents atteignent le même plafond

Ce dernier point est le plus important à retenir : **c'est le Hub qui borne, pas le nombre d'agents**. Ajouter des agents n'ajoute pas de débit de tâches — ajouter des Hubs, si (voir chapitre 18).

LES REQUÊTES VONT BIEN PLUS LOIN

Une requête (*ask*) est beaucoup moins coûteuse qu'une tâche : environ 1 500/s contre 11/s. Une tâche traverse la persistance, les garde-fous, le séquestre éventuel et le journal d'audit Merkle ; une requête, non. Si un flux de travail peut s'exprimer en requêtes plutôt qu'en tâches, il ira environ cent fois plus loin.

19.2 Le comportement sous saturation

```
intermesh hub --max-tasks-per-sec 8
```

Au-delà du budget, une soumission est **refusée**, jamais mise en file silencieusement. L'émetteur reçoit une erreur exploitable par une machine :

```
{
  "code": "HUB_SATURATED",
  "reason": "rate_limited",
  "retry_after_ms": 121,
  "queue_depth": 8,
  "in_flight": 8,
  "hub_tasks_per_sec_limit": 8,
  "task_id": "..."}
}
```

`reason` vaut `rate_limited`, `in_flight_limit` ou `task_queue_full` — trois murs distincts, trois réponses différentes. `retry_after_ms` indique le délai avant qu'un jeton soit réellement disponible : réessayer plus tôt ne fait qu'aggraver la congestion qu'on vient de rencontrer.

19.3 Réglages

Variable	Option CLI	Défaut	Borne
<code>INTERMESH_MAX_TASKS_PER_SEC</code>	<code>--max-tasks-per-sec</code>	10	Soumissions par seconde, à l'échelle du Hub entier
<code>INTERMESH_MAX_TASKS_IN_FLIGHT</code>	<code>--max-tasks-in-flight</code>	200	Tâches acceptées et pas encore terminées
<code>INTERMESH_MAX_TASK_QUEUE_DEPTH</code>	<code>--max-task-queue-depth</code>	100	Tâches acceptées mais pas encore prises en charge
<code>INTERMESH_BACKPRESSURE_ENABLED</code>	<code>--no-backpressure</code>	actif	Interrupteur général

Ces valeurs par défaut viennent directement de la mesure ci-dessus, pas d'une intuition. Un déploiement qui mesure autre chose sur son propre matériel doit les ajuster — c'est pour cela que ce sont des réglages, pas des constantes figées.

19.4 Observer la charge

La commande d'administration `hub.info` porte un bloc `load`, pour que la saturation soit visible **avant** que les clients ne commencent à tomber en dépassement de délai :

```
"load": {
  "in_flight": 12, "queue_depth": 4,
  "accepted_total": 100, "rejected_total": 440,
  "rejected_by_reason": {"rate_limited": 304, "task_queue_full": 136},
  "saturation": {"in_flight_pct": 6, "queue_depth_pct": 4}
}
```

`saturation` est la paire à surveiller en priorité. `rejected_by_reason` indique **quel** mur est heurté — et les trois murs appellent des réponses différentes.

19.5 Pièges connus

LE QUOTA PAR AGENT NE PROTÈGE PAS LE HUB

Le garde-fou existant (60 tâches/minute par agent, hérité du système de garde-fous) arrête un agent devenu fou. Il ne fait rien contre cent agents parfaitement sages, chacun à un tiers de son propre quota : cela représente 200 tâches/seconde, vingt fois ce qu'un Hub soutient. Les deux limites ne sont pas substituables l'une à l'autre.

- **Réessayer immédiatement après un refus** convertit une surcharge brève en surcharge soutenue. Respectez `retry_after_ms`.
- **Soumettre à un agent hors ligne** fait s'accumuler la tâche en file. Assez de tâches ainsi bloquées, et `queue_depth` atteint son plafond — le Hub refuse alors **tout**, y compris le travail destiné à des agents parfaitement disponibles. Vérifiez avec `discover` avant de soumettre.
- **Utiliser des tâches là où une requête suffirait** gaspille un facteur cent de débit disponible.

19.6 Ce que la contre-pression ne protège pas

Elle agit quand un message est **dépilé et analysé** par le Hub. Un client qui inonde plus vite que le Hub ne lit sa connexion s'accumule dans le transport réseau, hors de portée du mécanisme — limitez en amont, sur un relais, si des clients non fiables peuvent atteindre le Hub directement. Rien n'est non plus annulé rétroactivement : baisser le budget ralentit les admissions, il ne délesterait jamais ce qui est déjà en cours.

EXERCICE

Démarrez un Hub avec `--max-tasks-per-sec 2`. Envoyez rapidement une dizaine de tâches depuis un script, et observez les codes de refus reçus au-delà des deux premières.

CE QU'IL FAUT RETENIR

- Un Hub soutient environ 11 tâches/seconde, quel que soit le nombre d'agents connectés.
- Au-delà du budget configuré, une tâche est refusée explicitement, avec le délai avant réessai.
- Le quota par agent (anti-abus) et le budget global du Hub (anti-saturation) sont deux mécanismes complémentaires, pas interchangeables.
- **hub.info** expose la charge en temps réel, avant que la saturation ne devienne visible côté client.

20. Authentification mutuelle TLS (mTLS)

Le Hub sait déjà servir en `wss://` : le trafic est chiffré, et l'agent vérifie le certificat du Hub. L'authentification mutuelle ajoute l'inverse : le Hub vérifie aussi **le certificat du client** avant d'accepter la connexion.

20.1 Activer l'exigence côté Hub

```
intermesh hub \  
  --tls-cert hub.crt --tls-key hub.key \  
  --tls-client-ca autorite-interne.crt \  
  --require-api-key
```

Avec `--tls-client-ca`, un client sans certificat signé par cette autorité est refusé **pendant la poignée de main TLS elle-même** — avant le premier octet de protocole, donc avant tout enregistrement, toute clé d'API, et toute écriture au journal.

CELA NE REMPLACE PAS LES CLÉS D'API

Le certificat atteste que la **machine** fait partie du parc autorisé ; la clé d'API dit **quel agent** se connecte et avec quels rôles. Un porteur de certificat valide mais sans clé d'API reste refusé sous `--require-api-key`. « On a du mTLS, donc on peut alléger les clés » est la mauvaise lecture à éviter : les deux contrôles se cumulent.

20.2 Présenter un certificat en tant que Hub

Quand ce Hub appelle un pair ou un frère de grappe qui exige lui-même l'authentification mutuelle, il présente son propre certificat :

```
intermesh hub --mtls-cert client.crt --mtls-key client.key ...
```

20.3 Traçabilité

Le nom porté par le certificat client (son *common name*) est écrit dans le journal d'audit sur l'entrée d'enregistrement, sous `client_cert_cn`. Sans cette trace, l'authentification mutuelle refuserait bien les inconnus, mais sans jamais consigner qui elle a laissé entrer — la première question que pose une revue de sécurité. Seul le nom est journalisé, jamais le certificat lui-même : le journal d'audit ne porte aucune matière cryptographique.

20.4 Les liens de grappe aussi

Les liens entre Hubs frères d'une même grappe empruntent le même chemin TLS que les liens de fédération : une adresse `--cluster-url` en `wss://` est chiffrée, et présente un certificat client si l'authentification mutuelle est configurée sur le frère de destination.

CE QU'IL FAUT RETENIR

- `--tls-client-ca` exige un certificat client valide, vérifié avant tout protocole applicatif.
- Le certificat atteste la machine ; la clé d'API dit qui parle. Les deux se cumulent, ni ne se remplacent.
- Le nom du certificat client est journalisé pour la traçabilité, jamais le certificat lui-même.

21. Déploiement en réseau fermé et Docker durci

Ce chapitre s'adresse au cas où la machine n'a **aucune route vers Internet**, et où quelqu'un doit répondre de ce qui y tourne — une banque, un hôpital, un site sensible.

21.1 Ce que le logiciel ne fait pas

Vérifié en lisant le code source, pas affirmé sur parole :

- **Aucune vérification de licence, aucune télémétrie, aucune analytique, aucun rapport de plantage.** Il n'existe aucun code qui « appelle chez lui ». Un Hub coupé d'Internet se comporte exactement comme un Hub connecté.
- **Un seul appel sortant existe** : le pont HTTP (`bridge.py`) envoie vers l'URL que **vous** lui indiquez avec `--http`. Il ne va nulle part ailleurs.
- **Trois dépendances Python** : `websockets`, `PyJWT`, `cryptography`, plus `psycopg` si PostgreSQL est utilisé. Toutes installables depuis un miroir interne.

21.2 La pile Docker durcie

```
cp docker/hardened.env.example docker/hardened.env
chmod 600 docker/hardened.env          # puis remplir les trois secrets

docker compose --env-file docker/hardened.env \
    -f docker-compose.hardened.yml up -d
```

`--ENV-FILE` N'EST PAS FACULTATIF

`env_file:` injecte des variables *dans* les conteneurs, mais l'interpolation `${...}` du fichier `compose` lui-même est résolue par la CLI Docker contre son propre environnement. Sans le drapeau `--env-file`, la pile s'arrête en réclamant des variables qu'elle semble pourtant déjà porter.

Aspect	Comportement de la pile durcie
État	PostgreSQL, pas un fichier attaché au conteneur
Enregistrement	Clé d'API exigée partout, y compris depuis localhost
Console	Clé d'API exigée pour l'ouvrir ; refusée sans elle
TLS mutuel	Disponible via <code>--tls-client-ca</code> , désactivé sans autorité fournie
Saturation	Refus explicite au-delà du budget, jamais une file silencieuse
Réseau de la base	Interne — aucune route sortante, aucun port publié
Systèmes de fichiers	Lecture seule, avec des zones temporaires pour les rares chemins d'écriture
Privilèges	Sans droits root (uid 10001), toutes les capacités système retirées
Images	Épinglées par empreinte, jamais par étiquette
Ports publiés	Boucle locale uniquement

21.3 Vérifications, pas affirmations

À exécuter contre la pile réellement démarrée :

```
docker exec intermesh-hub id
# uid=10001(intermesh) gid=10001(intermesh)

docker exec intermesh-hub touch /app/test
# touch: cannot touch '/app/test': Read-only file system

docker exec intermesh-postgres getent hosts pypi.org
# (aucune réponse – la résolution DNS échoue)

nc -z 127.0.0.1 5432
# (le port de la base n'est pas publié)
```

21.4 Construire sans index de paquets

La construction de l'image exige un index `pip`. Sur une machine sans Internet, pointez vers votre miroir interne :

```
# Dans docker/Dockerfile.hub.hardened, à côté de pip install :
# --index-url https://pypi.interne.exemple/simple --trusted-host
pypi.interne.exemple
```

Ou construisez l'image sur une machine connectée et transférez-la :

```
docker save intermesh/hub:hardened | gzip > intermesh-hub.tar.gz
# transfert, puis sur la machine fermée :
gunzip -c intermesh-hub.tar.gz | docker load
```

Les images de base sont épinglées par empreinte cryptographique, donc l'image construite ailleurs est *l'image* qui tourne ici — c'est le but d'épingler par empreinte plutôt que par étiquette.

21.5 La console est le Control Plane, en version locale

La console (`sdk-python/intermesh/console/`) et le Control Plane hébergé servent la même application React. La version locale se distingue sur trois points seulement : pas de compte (connexion directe au Hub indiqué), une police système plutôt qu'une police téléchargée — ce qui permet de la **construire sans aucun réseau** —, et l'absence des routes qui appartiennent au site hébergé (devis, invitations, licences).

21.6 Ce qui manque encore pour un déploiement réglementé

Nommé plutôt que passé sous silence, car une revue de sécurité le trouvera de toute façon :

- **Pas d'annuaire LDAP ou Active Directory.** Les agents comme la console s'authentifient par clé d'API — adapté à des programmes, moins à des personnes : une clé est partagée, pas attribuable individuellement.
- **Pas d'intégration HSM.** La clé de signature est un secret d'environnement, pas un module matériel dédié.
- **Pas de nomenclature logicielle (SBOM), ni d'images signées, ni d'attestation de construction reproductible.** L'épinglage par empreinte est un plancher, pas un substitut.

CE QU'IL FAUT RETENIR

- Le logiciel ne contient aucun appel réseau caché ; un seul point de sortie explicite existe (`bridge.py --http`).
- La pile Docker durcie combine PostgreSQL, clés obligatoires, systèmes de fichiers en lecture seule et images épinglées.
- La console se construit sans aucun accès réseau grâce à la police système.
- LDAP, HSM et SBOM restent des manques assumés — à considérer avant un déploiement réglementé.

22. Bonnes pratiques et pièges connus

Synthèse des pièges rencontrés en conditions réelles, chacun documenté dans le code source ou les guides techniques du projet.

22.1 Développement

Piège	Remède
<code>await</code> au niveau module en Python	Erreur de syntaxe immédiate. Utilisez <code>agent.run()</code> ou enveloppez dans <code>async def main(): ...</code> + <code>asyncio.run(main())</code> .
Fonction synchrone lourde dans <code>from_callable</code>	Gèle toute la boucle asyncio. Enveloppez avec <code>asyncio.to_thread(...)</code> .
<code>@intermesh_service</code> remplace la fonction décorée	Elle n'est plus appellable comme une fonction ordinaire ensuite — normal, pas un bug.

22.2 Sécurité

Piège	Remède
<code>--dev-api-keys</code> en production	Jamais. Les valeurs figurent dans le dépôt public.
Relais inverse sans <code>--require-api-key</code>	Le contrôle d'origine ne protège plus rien : toute connexion paraît locale.
Pairage fédéré en <code>ws://</code> vers un hôte distant	Refusé par construction, hors <code>localhost</code> — n'utilisez <code>--allow-insecure-peering</code> qu'en réseau totalement maîtrisé.
mTLS considéré comme substitut des clés d'API	Les deux se cumulent ; un certificat valide sans clé reste refusé sous <code>--require-api-key</code> .

22.3 Capacité

Piège	Remède
Compter sur le quota par agent pour protéger le Hub	Cent agents sages dépassent vingt fois le plafond du Hub. Configurez la contre-pression globale.
Réessayer immédiatement après <code>HUB_SATURATED</code>	Aggrave la congestion. Respectez <code>retry_after_ms</code> .
Soumettre à un agent hors ligne	La tâche s'accumule en file jusqu'à saturer <code>queue_depth</code> pour tout le monde. Vérifiez avec <code>discover</code> d'abord.
Utiliser des tâches là où une requête suffit	Facteur cent de débit perdu inutilement.

22.4 Exploitation

Piège	Remède
Redémarrer un Hub sans redémarrer après renouvellement de certificat	Le certificat est lu au démarrage seulement — un renouvellement Certbot sans redémarrage sert un certificat expiré.
Changer <code>INTERMESH_HUB_SECRET</code> sans prévenir	Éjecte instantanément toute la flotte connectée — aucune rotation sans coupure n'existe à ce jour.
Exécutants non idempotents	Une tâche interrompue est réémise depuis le début, jamais reprise en cours — un exécutant qui n'est pas rejouable en toute sécurité créera des effets de bord dupliqués.

CE QU'IL FAUT RETENIR

- La grande majorité des pièges viennent d'un contrôle non cumulé avec un autre — clé d'API et mTLS, quota par agent et budget global.
- Chaque refus explicite du protocole (`SELF_DECLARED_REFUSED` , `HUB_SATURATED`) porte l'action corrective à prendre.

Annexe A — Référence complète de la CLI

```
intermesh hub
```

Option	Rôle
<code>--port</code>	Port d'écoute (défaut 8765).
<code>--org</code>	Organisation portée par ce Hub.
<code>--ephemeral-state</code>	État en mémoire, rien sur disque.
<code>--state-file</code>	Chemin de la base d'état SQLite.
<code>--state-dsn</code>	Base PostgreSQL, pour découpler l'état de la machine.
<code>--cluster-url</code>	Adresse à laquelle ce Hub est joignable par ses frères de grappe.
<code>--hub-id</code>	Identifiant de ce Hub dans la grappe (généré si absent).
<code>--allow-self-declared</code>	Accepte l'identité auto-déclarée depuis des connexions distantes.
<code>--require-api-key</code>	Exige une clé partout, y compris depuis <code>localhost</code> .
<code>--max-agents</code>	Plafond d'agents simultanés (0 = illimité).
<code>--ephemeral-secret</code>	Clé de signature JWT jetable.
<code>--secret-file</code>	Chemin du fichier de clé JWT.
<code>--snapshot-dir</code>	Dossier des instantanés d'état.
<code>--dev-api-keys</code>	Active des clés de démonstration — jamais en production.
<code>--peer</code>	<code>ORG=wss://host:port</code> — ouvre un lien de fédération.
<code>--tls-cert</code> / <code>--tls-key</code>	Certificat et clé pour servir en <code>wss://</code> .
<code>--tls-client-ca</code>	Autorité de certification des clients — active l'authentification mutuelle.
<code>--mtls-cert</code> / <code>--mtls-key</code>	Certificat que ce Hub présente à ses pairs/frères.
<code>--peer-ca</code>	Autorité de certification d'un pair (PKI privée).
<code>--allow-insecure-peering</code>	Autorise le pairage en clair vers un hôte distant.
<code>--max-tasks-per-sec</code>	Budget global de tâches par seconde (défaut 10).
<code>--max-tasks-in-flight</code>	Plafond de tâches acceptées non terminées (défaut 200).
<code>--max-task-queue-depth</code>	Profondeur de file avant refus (défaut 100).
<code>--no-backpressure</code>	Désactive le refus sous saturation.
<code>--approval-policy</code>	Politique de validation humaine (fichier JSON).
<code>--egress-policy</code>	Politique de filtrage de sortie (fichier JSON).

intermesh serve

Option	Rôle
<code>--name</code> / <code>-n</code>	Nom de l'agent (obligatoire).
<code>--exec</code>	Commande lancée par tâche (JSON entrée/sortie standard).
<code>--http</code>	URL recevant la tâche en <code>POST</code> JSON.
<code>--capability</code> / <code>-c</code>	Capacité annoncée, répétable.
<code>--org</code>	Organisation.
<code>--hub</code>	URL du Hub cible.
<code>--timeout</code>	Délai maximal accordé au programme externe.
<code>--no-encrypt</code>	Désactive le chiffrement E2E (débogage uniquement).

intermesh ping / ask / task

Commande	Syntaxe
ping	<code>intermesh ping <agent> [--hub URL] [--timeout N]</code>
ask	<code>intermesh ask <agent> '<JSON>' [--hub URL] [--timeout N]</code>
task	<code>intermesh task <agent> "<titre>" '<JSON>' [--hub URL] [--timeout N]</code>

intermesh discover

Option	Rôle
<code>--capability</code> / <code>-c</code>	Filtrer par capacité.
<code>--role</code> / <code>-r</code>	Filtrer par rôle.

intermesh snapshot

Sous-commande	Rôle
create	Créer un instantané (<code>--name</code>).
list	Lister les instantanés existants.
restore	Restaurer un instantané par son nom.
delete	Supprimer un instantané.

Options communes : `--api-key` (clé admin), `--encrypt` (chiffre/déchiffre le fichier).

`intermesh keygen`

Option	Rôle
<code>--out</code> / <code>-o</code>	Chemin de la clé privée générée (la publique reçoit l'extension <code>.pub</code>).

`intermesh dashboard / security-check / docs`

Sans option obligatoire. `dashboard` lance la console locale sur le port 8080 ; `security-check` audite l'installation locale ; `docs` affiche la documentation embarquée.

Annexe B — Glossaire

Agent

Tout programme connecté au Hub, quel que soit son langage, capable d'envoyer et de recevoir des messages, requêtes ou tâches.

Capacité

Étiquette libre décrivant ce qu'un agent sait faire, utilisée pour la découverte sans connaître son nom exact.

Chiffrement de bout en bout (E2E)

RSA-2048-OAEP + AES-256-GCM. Le Hub route le contenu chiffré sans pouvoir le lire.

Console d'administration

Interface web locale (`intermesh dashboard`), livrée dans le paquet, sans dépendance externe ni compte.

Contre-pression (backpressure)

Mécanisme de refus explicite d'une soumission de tâche quand le Hub dépasse son budget configuré, plutôt qu'une mise en file silencieuse.

Control Plane

Application web hébergée sur `intermesh.site`, avec comptes et gestion d'équipe — distincte de la console locale.

Egress (politique de sortie)

Ensemble de règles JSON contrôlant ce qui a le droit de quitter une organisation vers un partenaire fédéré : suppression de champ, occultation de motif, ou blocage complet.

Fédération

Lien entre deux Hubs de deux organisations différentes, établi par échange de clés publiques et vérification de signature Ed25519 — jamais de secret partagé.

Grappe (cluster)

Plusieurs Hubs d'une **même** organisation, partageant un état PostgreSQL et une clé de signature, formant un seul maillage.

Hub

Le serveur de coordination : registre des agents, routage des messages, authentification, journal d'audit.

Identité auto-déclarée

Enregistrement sans clé d'API, où l'agent choisit lui-même son organisation, ses rôles et ses permissions — accepté par défaut seulement depuis `localhost`.

JWT (JSON Web Token)

Jeton signé par la clé Ed25519 du Hub, porté par chaque message après l'enregistrement.

Journal d'audit (chaîné par Merkle)

Historique infalsifiable des événements du Hub : toute modification rétroactive d'une entrée brise la chaîne, détectée au redémarrage.

mTLS (authentification mutuelle TLS)

Le Hub vérifie le certificat du client, en plus de présenter le sien — se cumule aux clés d'API, ne les remplace pas.

Tâche

Unité de travail déléguée, avec un cycle de vie `pending → running → completed/failed` et une reprise automatique en cas d'interruption.

Wss:// / ws://

`wss://` est le WebSocket chiffré par TLS, obligatoire pour tout Hub distant. `ws://` est le WebSocket en clair, réservé à `localhost`.

FIN DE LA FORMATION

Cette formation couvre l'installation, l'usage quotidien des deux SDK officiels, l'intégration de frameworks existants, l'orchestration multi-agents, et le déploiement en production — fédération, grappe, capacité, authentification mutuelle et réseau fermé. Pour toute question non couverte ici, la documentation embarquée ([intermesh docs](#)) et le dépôt source font foi en cas de divergence, le protocole évoluant plus vite qu'un document imprimé.